VPS DEPLOYMENT GUIDE

Complete reference for running the admin server on a Linux VPS (Debian/Ubuntu): connection, directory layout, service format, connection URL formats, environment variables, database, firewall, TLS, and troubleshooting.

The VPS hosts the Django + Daphne admin server behind nginx. Client agents connect over HTTPS + WebSocket to register, stream heartbeats, receive scheduled scans, and report scan data. A shared Supabase PostgreSQL database is used in production so multiple deployment targets (VPS and Vercel) see the same data.

Generated September 08, 2026

1. Overview

Key	Description
Web server / reverse proxy	nginx (public TCP 80/443)
App server (ASGI)	Daphne via `admin/main.py --asgi` (binds `127.0.0.1:8000`)
Database	Supabase PostgreSQL (via `DATABASE_URL`)
WebSocket	Django Channels served by Daphne, proxied through nginx `/ws/`
Discovery	UDP broadcast on port 45000 (LAN) + Supabase cloud registry (WAN)
TLS	certbot / Let's Encrypt
Process manager	systemd (`system-scanner-admin.service`)

2. Prerequisites

- A VPS reachable from the internet (Debian/Ubuntu recommended)
- DNS **A record** pointing your domain at the VPS public IP
- Python 3.10+ available on the server
- Firewall open for TCP 80/443 and UDP 45000
- A Supabase project (PostgreSQL) with the schema installed (`setup_new_supabase.sql`)
- A populated `.env` next to the repo on the server

3. Connecting to the VPS (SSH)

Standard password / PEM-key SSH from any machine:

```
# With password (or default key)
ssh root@YOUR.VPS.IP
# With a PEM private key (AWS / DigitalOcean / GCP style)
ssh -i /path/to/key.pem ubuntu@YOUR.VPS.IP
# With a specific port
ssh -p 2222 root@YOUR.VPS.IP
```

Copy the project to the server (the install script expects it at `/opt/scanner-admin`):

```
# With scp
scp -r ./admin-client root@YOUR.VPS.IP:/opt/scanner-admin
# With rsync
rsync -avP ./admin-client/ root@YOUR.VPS.IP:/opt/scanner-admin/
```

APP_DIR is `/opt/scanner-admin` by default. Override it with the APP_DIR environment variable.

4. Server Layout / Format

Once deployed, the server looks like this:

```
/opt/scanner-admin # APP_DIR (repo)
|-- .env # secrets + configuration (required)
```

```
|-- requirements.txt
|-- admin/
| |-- main.py # entrypoint (Django + Daphne + discovery)
| |-- django_admin/
| | |-- settings.py
| | `-- asgi.py # Channels ASGI app
| `-- scanner_api/ # models, views, supabase_client, ...
|-- venv/ # Python virtualenv
`-- deploy/
|-- install.sh # provisioning script (run as root)
|-- register_server.sh # manual cloud-discovery registration
|-- nginx/scanner.conf # reverse proxy template
`-- systemd/system-scanner-admin.service
/etc/systemd/system/system-scanner-admin.service # app is a systemd service
/etc/nginx/sites-available/scanner.conf # nginx site (symlinked into sites-enabled)
```

5. Environment Variables (.env Format)

Copy `.env.template` to the server as `.env`. All values are read by the systemd unit via `EnvironmentFile=/opt/scanner-admin/.env`.

```
# Django
DJANGO_SECRET_KEY="django-insecure-change-me-in-production-abc123"
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS="*"
# Supabase PostgreSQL (Django ORM)
# NOTE: direct db.<ref>.supabase.co is IPv6-only. For IPv4-only networks use
# the Supavisor shared pooler (session mode, port 5432), username postgres.<ref>.
DATABASE_URL="postgresql://postgres.<project-ref>:<password>@aws-0-<region>.pooler.supabase.com:
6543/postgres?sslmode=require"
# Supabase Client (Direct API queries)
SUPABASE_URL="https://<project-ref>.supabase.co"
SUPABASE_SERVICE_KEY="<service-role-jwt>"
SUPABASE_JWT_SECRET="<jwt-secret>"
```

Key	Description
DJANGO_SECRET_KEY	Django secret used to sign sessions / tokens
DJANGO_DEBUG	Set `False` in production
DJANGO_ALLOWED_HOSTS	Hosts allowed to serve the app (* with `--host 0.0.0.0`)
DATABASE_URL	Postgres connection string for the Django ORM
SUPABASE_URL	Supabase REST host used by the client registry queries
SUPABASE_SERVICE_KEY	Service-role key for direct Supabase API calls
SUPABASE_JWT_SECRET	JWT secret for verifying Supabase tokens

Never commit .env to git. .env is gitignored; only `.env.template` is tracked.

6. Server Components

6.1 systemd service

```
[Unit]
Description=System Scanner Pro Admin Server (Daphne ASGI)
After=network-online.target
Wants=network-online.target
[Service]
Type=simple
User=scanner
Group=scanner
WorkingDirectory=/opt/scanner-admin
Environment=DJANGO_SETTINGS_MODULE=django_admin.settings
EnvironmentFile=/opt/scanner-admin/.env
# Daphne binds the private backend port; nginx proxies public 80/443 -> 8000.
ExecStart=/opt/scanner-admin/venv/bin/python admin/main.py --host 127.0.0.1 --port 8000
--asgi --domain scanner.example.com
Restart=on-failure
RestartSec=5
KillSignal=SIGINT
[Install]
WantedBy=multi-user.target
```

Key	Description
--host 127.0.0.1	Bind only the private backend port (nginx proxies it)
--port 8000	Backend port nginx forwards to
--asgi	Serve with Daphne (WebSocket support)
--domain scanner.example.com	Registers `https://<domain>` in the Supabase cloud registry for WAN discovery

6.2 nginx

Templated from `deploy/nginx/scanner.conf`. Responsibilities: HTTP → HTTPS redirect on port 80, TLS termination on 443 (certbot certificates), reverse proxy REST + static to `http://127.0.0.1:8000`, and WebSocket proxying (`/ws/`) with Upgrade headers.

Format	Purpose
`/`	REST API + dashboard (proxy to 127.0.0.1:8000)
`/ws/`	WebSocket (real-time dashboard, agent commands); `proxy_read_timeout 86400` with Upgrade/Connection headers
`/static/`	Static assets (optional; whitenoise also serves them)

6.3 Install script

```
# From the server, with the repo at /opt/scanner-admin
cd /opt/scanner-admin
DOMAIN=scanner.example.com ./deploy/install.sh
```

Performs: package install → service user creation → venv + pip install → systemd unit install → nginx site install → certbot TLS. Re-run it any time to re-apply configuration.

7. Connection URL Formats

7.1 Admin dashboard

Format	Purpose
`https://<domain>/`	Dashboard
`https://<domain>/login/`	Admin login (default: admin / admin123)

7.2 Client connect URL (paste-to-connect)

Each admin gets a unique connect URL that clients accept as a command-line argument:

```
https://<domain>/connect/<username>/<company>/
```

```
python client/main.py https://scanner.example.com/connect/admin/mycompany/
SystemScannerClient.exe https://scanner.example.com/connect/admin/mycompany/
# Alternative - base admin URL (clients auto-parse it)
python client/main.py https://scanner.example.com
```

> The connect page is public; anyone visiting it sees connection instructions. Clients still start as **pending** and require explicit admin approval.

7.3 WebSocket endpoints (real-time)

Format	Purpose
`wss://<domain>/ws/dashboard/`	Admin dashboard real-time updates
`wss://<domain>/ws/agent/<agent_id>/`	Agent command / control channel

In production use `wss://` (TLS). In development on plain HTTP use `ws://`.

7.4 REST API base

All API routes live under `https://<domain>/api/...` (registration, heartbeat, scan submit, monitoring, scheduling, JWT, reports). See the README 'API Endpoints' table for the full list.

7.5 Ports

Format	Purpose
`80 / TCP`	HTTP → HTTPS redirect
`443 / TCP`	HTTPS dashboard + API + WebSocket
`8000 / TCP`	Private backend (loopback only - do not expose this to the internet)
`45000 / UDP`	LAN auto-discovery (admin broadcasts, clients listen)

7.6 Database connection string format

```
postgresql://postgres.<project-ref>:<password>@<region>.pooler.supabase.com:6543/postgres?sslmode=require
```

Key	Description
User	`postgres.<project-ref>` (session mode via Supavisor)
Host	`aws-0-<region>.pooler.supabase.com` (IPv4-friendly pooler)
Port	`6543` (session mode; transaction mode uses 5432)

Key	Description
Database	`postgres` (default database name)
Param	`sslmode=require` (always require TLS)

8. Firewall

Open on the VPS:

```
ufw allow 80/tcp # HTTP
ufw allow 443/tcp # HTTPS
ufw allow 45000/udp # client auto-discovery
ufw enable
```

9. Client-to-Server Resolution Order

When a client starts, it finds the admin server in this order:

```
1. Explicit connect URL / CLI argument (https://<domain>/connect/.../)
2. ADMIN_SERVER_URL environment variable
3. Supabase cloud registry (the VPS registers itself on startup via --domain)
4. Cached client_config.json
5. UDP LAN discovery (port 45000, same network)
6. Manual prompt
```

10. Cloud Discovery Registration

Automatic (normal path): the systemd unit passes `--domain scanner.example.com`, and `admin/main.py` calls `register_server_in_registry(domain, 443, "https")` on startup.

Manual (fallback):

```
cd /opt/scanner-admin
./deploy/register_server.sh scanner.example.com # https, port 443
./deploy/register_server.sh 1.2.3.4 80 http # IP with explicit port/protocol
```

Only one production admin should be registered in the cloud registry at a time. A local/dev panel must NOT call `--cloud`, or clients will bounce between two 'admin systems' and never see the correct server.

11. Operational Tasks

Start / stop / restart

```
systemctl start system-scanner-admin
systemctl stop system-scanner-admin
systemctl restart system-scanner-admin
systemctl --no-pager --full status system-scanner-admin
```

Logs

```
journalctl -u system-scanner-admin -f # follow app logs
tail -f /var/log/nginx/access.log # nginx access log
tail -f /var/log/nginx/error.log # nginx error log
```

Health check

```
curl -I https://<domain>/__health # expect HTTP 200
```

Apply a code update

```
cd /opt/scanner-admin
git pull # pull new source
venv/bin/pip install -r requirements.txt # new dependencies
python admin/manage.py migrate # apply DB migrations
systemctl restart system-scanner-admin # restart the service
nginx -t && systemctl reload nginx # reload nginx if config changed
```

Renew TLS

```
certbot renew --dry-run # test the Let's Encrypt auto-renew timer
systemctl reload nginx
```

12. Troubleshooting

Symptom	Fix
Client says 'Checking...' forever	The client registered but needs **admin approval** on the dashboard. Check https://<domain>/ for a pending client.
Client can't reach the server	Confirm the DNS A record points to the VPS IP; open TCP 80/443; check `systemctl status system-scanner-admin`.
Cloud discovery fails at startup	.env missing or misconfigured SUPABASE_URL / SUPABASE_SERVICE_KEY; run `./deploy/register_server.sh`.
WebSocket won't connect	Ensure the nginx /ws/ block has Upgrade/Connection: upgrade headers; use wss:// in production.
502 Bad Gateway	Daphne is down (`systemctl restart system-scanner-admin`) or nginx isn't proxying to 127.0.0.1:8000.
Scheduler warning on startup	Run `python admin/manage.py migrate`.
DB connection errors	The direct db.<ref>.supabase.co host is IPv6-only - switch DATABASE_URL to the Supavisor pooler (see section 7.6).
Two 'admin systems' / clients bouncing	A dev panel called --cloud and registered itself. Only the single production admin should register in the registry.